Task-1:-Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for deploying a sample Node.js application using AWS modern container services. This document serves as a guide for DevOps engineers and stakeholders to ensure the infrastructure is scalable, secure, and follows AWS best practices.

1.2 Scope

- **In-Scope:** Containerization of the `docker-nodejs-sample` app, ECR repository management, ECS Fargate cluster setup, Application Load Balancer (ALB) configuration, and VPC networking.
- **Out-of-Scope:** Setting up a CI/CD pipeline (Jenkins/GitHub Actions), SSL/TLS certificate integration (HTTPS), and persistent database migration.

1.3 Definitions, Acronyms, and Abbreviations

- **ECR:** Elastic Container Registry
- **ECS:** Elastic Container Service
- **Fargate:** AWS serverless compute engine for containers.
- **ALB:** Application Load Balancer
- **VPC:** Virtual Private Cloud
- **Task Definition:** A JSON file that describes one or more containers that form your application.

1.4 References

- AWS Documentation: Amazon ECS on AWS Fargate
- Docker Documentation: Build and Push Images
- GitHub Repository: pushkar-iamops/docker-nodejs

1.5 Overview

This document covers the project objectives, specific functional use cases for deployment, security and performance requirements, and a risk analysis of the cloud infrastructure setup.

---

2. Project Overview

2.1 Project Objectives

- To migrate a local Node.js application to a cloud-native containerized environment.
- To implement a serverless architecture using ECS Fargate to reduce operational overhead.

2.2 Business Goals

- **Scalability:** Ensure the application can handle traffic spikes by running multiple concurrent tasks.
- **Cost Efficiency:** Use Fargate's pay-as-you-go model to eliminate costs associated with idle EC2 instances.

2.3 Background

The current application exists as a Git repository. To meet modern production standards, it must be containerized to ensure consistency across different environments (Development, Testing, and Production).

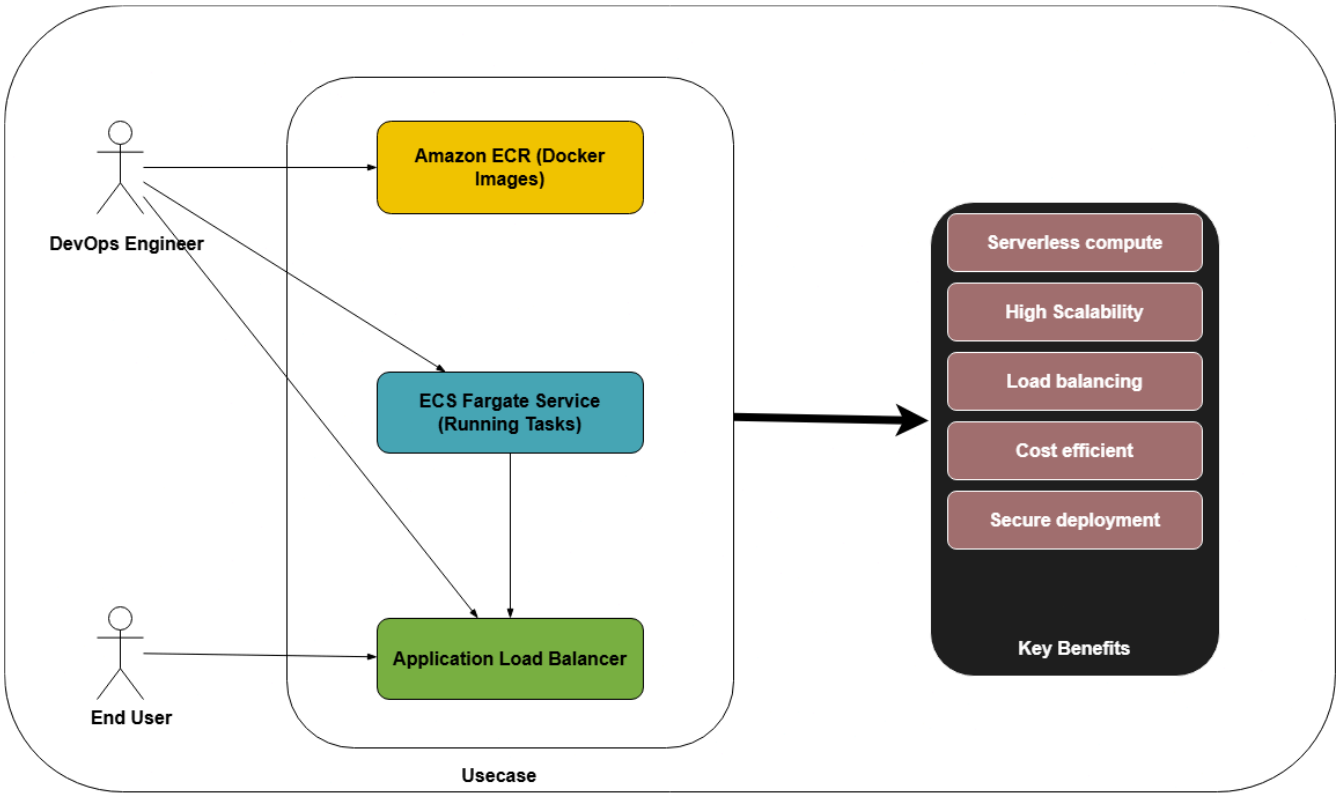
2.4 Constraints

- **Technology:** Must use AWS-specific services (ECS, ECR).
- **Resources:** Limited to 256 **vCPU** and 512 **MB RAM** per task.
- **Connectivity:** Must be accessible via public internet over HTTP (Port 80).

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Image Management and Deployment

- **Description:** The engineer builds the Docker image and makes it available for the AWS cloud environment.
- **Actors:** DevOps Engineer, AWS CLI, ECR.
- **Preconditions:** Local Docker engine is running; AWS CLI is configured with appropriate IAM permissions.
- **Postconditions:** Docker image is securely stored in ECR with a unique tag.
- **Flow of Events:**
  a. Clone the repository from GitHub.
  b. Build the Docker image locally using `docker build`.
  c. Authenticate the Docker client to the ECR registry.
  d. Tag the image and push it to the ECR repository.

3.2.2 Use Case 2: Orchestration and Access

- **Description:** Deploy the application to ECS and provide a single point of entry via an ALB.

- **Actors:** ECS Scheduler, Application Load Balancer, End User.
- **Preconditions:** Image exists in ECR; VPC and Subnets are configured.
- **Postconditions:** App is reachable via ALB DNS.
- **Flow of Events:**
  a. Create an ECS Cluster.
  b. Register a Task Definition referencing the ECR Image URI.
  c. Create an ECS Service with a desired task count of 2.
  d. The ALB registers the tasks as healthy targets.
- **Alternative Flows:**
  a. **Task Failure:** ECS automatically restarts a failed task to maintain the desired count of 2.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- **Availability:** The application should maintain 99.9% uptime by distributing tasks across multiple Availability Zones.
- **Elasticity:** The ALB should handle varying request rates without manual intervention.

4.2 Security Requirements

- **Network Isolation:** ECS tasks reside in a private security group that only accepts traffic from the ALB's security group.
- **IAM Roles:** Use the "Least Privilege" principle for the ECS Task Execution Role.

4.3 Usability Requirements

- The final output should be accessible via a standard web browser using a simplified DNS URL provided by the ALB.

4.4 Reliability Requirements

- The service must automatically replace unhealthy containers (Self-healing).

---

5. Assumptions and Dependencies

5.1 Assumptions

- It is assumed the user has an active AWS account with sufficient quotas for ECS and ECR.
- It is assumed the Node.js application code is functional and contains a valid `Dockerfile`.

5.2 Dependencies

- **AWS Infrastructure:** Availability of ECR and ECS services in the chosen region.
- **Docker Hub:** Required if the base image in the `Dockerfile` is pulled from a public registry.

---

6. Risk Analysis

6.1 Risk Identification

1. **Cost Overrun:** Leaving the ALB or Fargate tasks running indefinitely.
2. **Security Breach:** Leaving Port 80 open to the world on the Task level rather than the ALB level.

6.2 Risk Mitigation

1. **Cost:** Set up AWS Budgets and delete resources immediately after the task is verified.
2. **Security:** Implement strict Security Group rules as defined in Section 4.2.

---

7. Glossary

- **Cluster:** A logical grouping of tasks or services.
- **Inbound Rule:** A firewall rule determining what traffic can enter a resource.

---

8. Appendices

- **Project GitHub:** `pushkar-iamops/docker-nodejs`
- **Local Test Command:** `docker run -p 8080:8080 [image_name]`

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____